

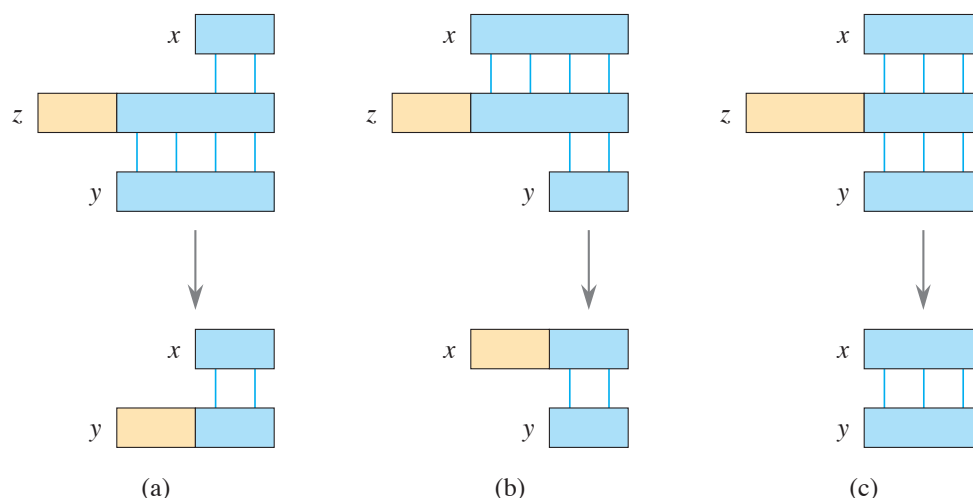


Figure 32.2 A graphical proof of Lemma 32.1. Suppose that $x \sqsubseteq z$ and $y \sqsubseteq z$. The three parts of the figure illustrate the three cases of the lemma. Vertical lines connect matching regions (shown in blue) of the strings. **(a)** If $|x| \leq |y|$, then $x \sqsubseteq y$. **(b)** If $|x| \geq |y|$, then $y \sqsubseteq x$. **(c)** If $|x| = |y|$, then $x = y$.

strings. The 0-length *empty string*, denoted ε , also belongs to Σ^* . The length of a string x is denoted $|x|$. The *concatenation* of two strings x and y , denoted xy , has length $|x| + |y|$ and consists of the characters from x followed by the characters from y .

A string w is a *prefix* of a string x , denoted $w \sqsubseteq x$, if $x = wy$ for some string $y \in \Sigma^*$. Note that if $w \sqsubseteq x$, then $|w| \leq |x|$. Similarly, a string w is a *suffix* of a string x , denoted $w \sqsupseteq x$, if $x = yw$ for some $y \in \Sigma^*$. As with a prefix, $w \sqsupseteq x$ implies $|w| \leq |x|$. For example, $ab \sqsubseteq abcca$ and $cca \sqsupseteq abcca$. A string w is a *proper prefix* of x if $w \sqsubseteq x$ and $|w| < |x|$, and likewise for a *proper suffix*. The empty string ε is both a suffix and a prefix of every string. For any strings x and y and any character a , we have $x \sqsubseteq y$ if and only if $xa \sqsubseteq ya$. The $\sqsubseteq$ and $\sqsupseteq$ relations are transitive. The following lemma will be useful later.

Lemma 32.1 (Overlapping-suffix lemma)

Suppose that x , y , and z are strings such that $x \sqsubseteq z$ and $y \sqsubseteq z$. If $|x| \leq |y|$, then $x \sqsubseteq y$. If $|x| \geq |y|$, then $y \sqsubseteq x$. If $|x| = |y|$, then $x = y$.

Proof See Figure 32.2 for a graphical proof. ■

For convenience, denote the k -character prefix $P[1:k]$ of the pattern $P[1:m]$ by $P[:k]$. Thus, we can write $P[:0] = \varepsilon$ and $P[:m] = P = P[1:m]$. Similarly, denote the k -character prefix of the text T by $T[:k]$. Using this notation, we

can state the string-matching problem as that of finding all shifts s in the range $0 \leq s \leq n - m$ such that $P \sqsubseteq T[s : s + m]$.

Our pseudocode allows two equal-length strings to be compared for equality as a primitive operation. If the strings are compared from left to right and the comparison stops when a mismatch is discovered, we assume that the time taken by such a test is a linear function of the number of matching characters discovered. To be precise, the test “ $x == y$ ” is assumed to take $\Theta(t)$ time, where t is the length of the longest string z such that $z \sqsubseteq x$ and $z \sqsubseteq y$.

32.1 The naive string-matching algorithm

The NAIVE-STRING-MATCHER procedure finds all valid shifts using a loop that checks the condition $P[1 : m] = T[s + 1 : s + m]$ for each of the $n - m + 1$ possible values of s .

```

NAIVE-STRING-MATCHER( $T, P, n, m$ )
1  for  $s = 0$  to  $n - m$ 
2      if  $P[1 : m] == T[s + 1 : s + m]$ 
3          print “Pattern occurs with shift”  $s$ 

```

Figure 32.3 portrays the naive string-matching procedure as sliding a “template” containing the pattern over the text, noting for which shifts all of the characters on the template equal the corresponding characters in the text. The **for** loop of lines 1–3 considers each possible shift explicitly. The test in line 2 determines whether the current shift is valid. This test implicitly loops to check corresponding character positions until all positions match successfully or a mismatch is found. Line 3 prints out each valid shift s .

Procedure NAIVE-STRING-MATCHER takes $O((n - m + 1)m)$ time, and this bound is tight in the worst case. For example, consider the text string a^n (a string of n a ’s) and the pattern a^m . For each of the $n - m + 1$ possible values of the shift s , the implicit loop on line 2 to compare corresponding characters must execute m times to validate the shift. The worst-case running time is thus $\Theta((n - m + 1)m)$, which is $\Theta(n^2)$ if $m = \lfloor n/2 \rfloor$. Because it requires no preprocessing, NAIVE-STRING-MATCHER’s running time equals its matching time.

NAIVE-STRING-MATCHER is far from an optimal procedure for this problem. Indeed, this chapter will show that the Knuth-Morris-Pratt algorithm is much better in the worst case. The naive string-matcher is inefficient because it entirely ignores information gained about the text for one value of s when it considers other values of s . Such information can be quite valuable, however. For example, if $P = \text{aaab}$

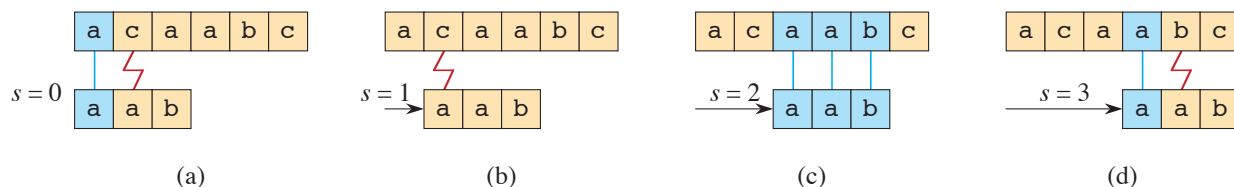


Figure 32.3 The operation of the NAIVE-STRING-MATCHER procedure for the pattern $P = aab$ and the text $T = acaabc$. Imagine the pattern P as a template that slides next to the text. (a)–(d) The four successive alignments tried by the naive string matcher. In each part, vertical lines connect corresponding regions found to match (shown in blue), and a red jagged line connects the first mismatched character found, if any. The algorithm finds one occurrence of the pattern, at shift $s = 2$, shown in part (c).

and $s = 0$ is valid, then none of the shifts 1, 2, or 3 are valid, since $T[4] = b$. The following sections examine several ways to make effective use of this sort of information.

Exercises

32.1-1

Show the comparisons the naive string matcher makes for the pattern $P = 0001$ in the text $T = 000010001010001$.

32.1-2

Suppose that all characters in the pattern P are different. Show how to accelerate NAIVE-STRING-MATCHER to run in $O(n)$ time on an n -character text T .

32.1-3

Suppose that pattern P and text T are *randomly* chosen strings of length m and n , respectively, from the d -ary alphabet $\Sigma_d = \{0, 1, \dots, d-1\}$, where $d \geq 2$. Show that the *expected* number of character-to-character comparisons made by the implicit loop in line 2 of the naive algorithm is

$$(n - m + 1) \frac{1 - d^{-m}}{1 - d^{-1}} \leq 2(n - m + 1)$$

over all executions of this loop. (Assume that the naive algorithm stops comparing characters for a given shift once it finds a mismatch or matches the entire pattern.) Thus, for randomly chosen strings, the naive algorithm is quite efficient.

32.1-4

Suppose that the pattern P may contain occurrences of a *gap character* $\diamond$ that can match an *arbitrary* string of characters (even one of 0 length). For example, the pattern $ab\diamond ba\diamond c$ occurs in the text $cabccbacbacab$ as

c ab cc ba cba c ab
 ab ◇ ba ◇ c

and as

c ab ccbac ba c ab.
 ab ◇ ba ◇ c

The gap character may occur an arbitrary number of times in the pattern but not at all in the text. Give a polynomial-time algorithm to determine whether such a pattern P occurs in a given text T , and analyze the running time of your algorithm.

32.2 The Rabin-Karp algorithm

Rabin and Karp proposed a string-matching algorithm that performs well in practice and that also generalizes to other algorithms for related problems, such as two-dimensional pattern matching. The Rabin-Karp algorithm uses $\Theta(m)$ preprocessing time, and its worst-case running time is $\Theta((n - m + 1)m)$. Based on certain assumptions, however, its average-case running time is better.

This algorithm makes use of elementary number-theoretic notions such as the equivalence of two numbers modulo a third number. You might want to refer to Section 31.1 for the relevant definitions.

For expository purposes, let's assume that $\Sigma = \{0, 1, 2, \dots, 9\}$, so that each character is a decimal digit. (In the general case, you can assume that each character is a digit in radix- d notation, so that it has a numerical value in the range 0 to $d - 1$, where $d = |\Sigma|$.) You can then view a string of k consecutive characters as representing a length- k decimal number. For example, the character string 31415 corresponds to the decimal number 31,415. Because we interpret the input characters as both graphical symbols and digits, it will be convenient in this section to denote them as digits in standard text font.

Given a pattern $P[1:m]$, let p denote its corresponding decimal value. In a similar manner, given a text $T[1:n]$, let t_s denote the decimal value of the length- m substring $T[s + 1:s + m]$, for $s = 0, 1, \dots, n - m$. Certainly, $t_s = p$ if and only if $T[s + 1:s + m] = P[1:m]$, and thus, s is a valid shift if and only if $t_s = p$. If you could compute p in $\Theta(m)$ time and all the t_s values in a total of $\Theta(n - m + 1)$ time,² then you could determine all valid shifts s in $\Theta(m) + \Theta(n - m + 1) = \Theta(n)$

² We write $\Theta(n - m + 1)$ instead of $\Theta(n - m)$ because s takes on $n - m + 1$ different values. The “+1” is significant in an asymptotic sense because when $m = n$, computing the lone t_s value takes $\Theta(1)$ time, not $\Theta(0)$ time.

time by comparing p with each of the t_s values. (For the moment, let's not worry about the possibility that p and the t_s values might be very large numbers.)

Indeed, you can compute p in $\Theta(m)$ time using Horner's rule (see Problem 2-3):

$$p = P[m] + 10 \left(P[m-1] + 10 \left(P[m-2] + \cdots + 10(P[2] + 10P[1]) \cdots \right) \right).$$

Similarly, you can compute t_0 from $T[1:m]$ in $\Theta(m)$ time.

To compute the remaining values $t_1, t_2, \dots, t_{n-m}$ in $\Theta(n-m)$ time, observe that you can compute t_{s+1} from t_s in constant time, since

$$t_{s+1} = 10(t_s - 10^{m-1}T[s+1]) + T[s+m+1]. \quad (32.1)$$

Subtracting $10^{m-1}T[s+1]$ removes the high-order digit from t_s , multiplying the result by 10 shifts the number left by one digit position, and adding $T[s+m+1]$ brings in the appropriate low-order digit. For example, suppose that $m = 5$, $t_s = 31415$, and the new low-order digit is $T[s+5+1] = 2$. The high-order digit to remove is $T[s+1] = 3$, and so

$$\begin{aligned} t_{s+1} &= 10(31415 - 10000 \cdot 3) + 2 \\ &= 14152 \end{aligned}$$

If you precompute the constant 10^{m-1} (which you can do in $O(\lg m)$ time using the techniques of Section 31.6, although for this application a straightforward $O(m)$ -time method suffices), then each execution of equation (32.1) takes a constant number of arithmetic operations. Thus, you can compute p in $\Theta(m)$ time, and you can compute all of $t_0, t_1, \dots, t_{n-m}$ in $\Theta(n-m+1)$ time. Therefore, you can find all occurrences of the pattern $P[1:m]$ in the text $T[1:n]$ with $\Theta(m)$ preprocessing time and $\Theta(n-m+1)$ matching time.

This scheme works well if P is short enough and the alphabet Σ is small enough that arithmetic operations on p and t_s take constant time. But what if P is long, or if the size of Σ means that instead of powers of 10 in equation (32.1) you have to use powers of a larger number (such as powers of 256 for the extended ASCII character set)? Then the values of p and t_s might be too large to work with in constant time. Fortunately, this problem can be solved, as Figure 32.4 shows: compute p and the t_s values modulo a suitable modulus q . You can compute p modulo q in $\Theta(m)$ time and all the t_s values modulo q in $\Theta(n-m+1)$ time. With $|\Sigma| = 10$, if you choose the modulus q as a prime such that $10q$ just fits within one computer word, then you can perform all the necessary computations with single-precision arithmetic. In general, with a d -ary alphabet $\{0, 1, \dots, d-1\}$, choose q so that dq fits within a computer word and adjust the recurrence equation (32.1) to work modulo q , so that it becomes

$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q, \quad (32.2)$$

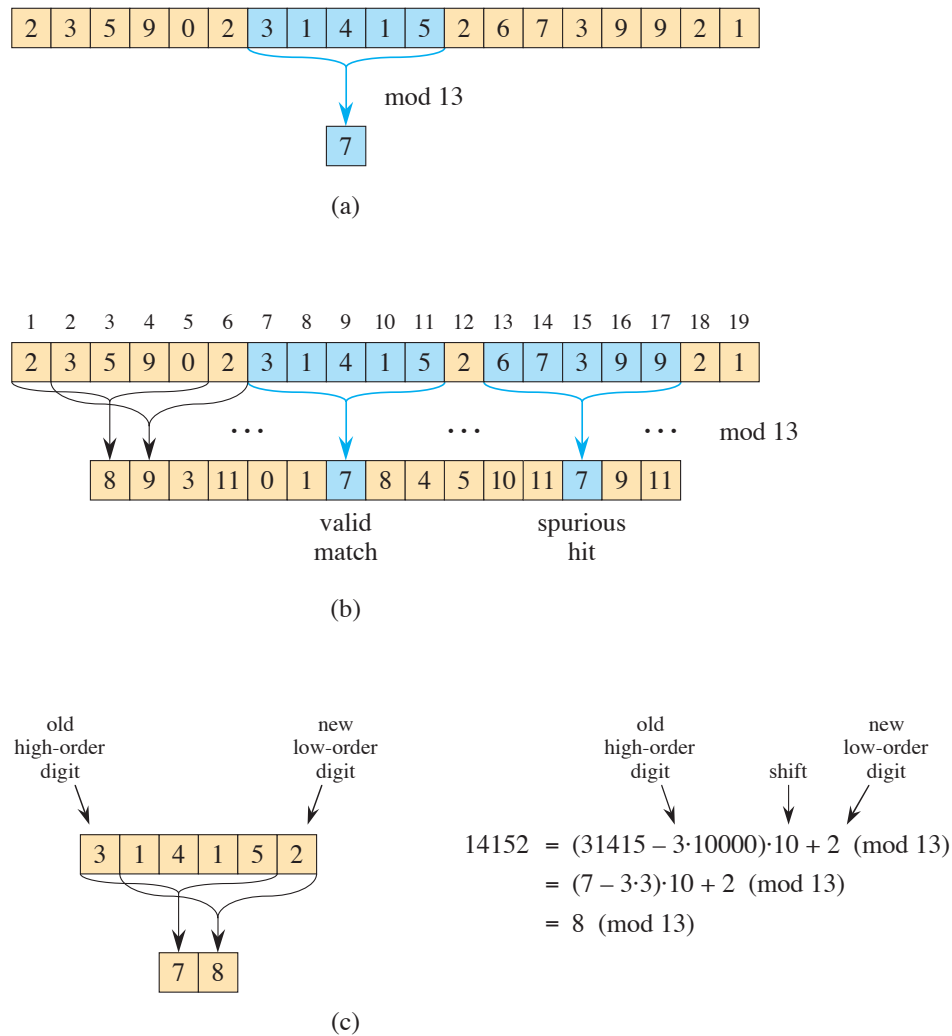


Figure 32.4 The Rabin-Karp algorithm. Each character is a decimal digit. Values are computed modulo 13. **(a)** A text string. A window of length 5 is shaded blue. The numerical value of the blue number, computed modulo 13, yields the value 7. **(b)** The same text string with values computed modulo 13 for each possible position of a length-5 window. Assuming the pattern $P = 31415$, look for windows whose value modulo 13 is 7, since $31415 = 7 \pmod{13}$. The algorithm finds two such windows, shaded blue in the figure. The first, beginning at text position 7, is indeed an occurrence of the pattern. The second window, beginning at text position 13, is a spurious hit. **(c)** How to compute the value for a window in constant time, given the value for the previous window. The first window has value 31415. Dropping the high-order digit 3, shifting left (multiplying by 10), and then adding in the low-order digit 2 gives the new value 14152. Because all computations are performed modulo 13, the value for the first window is 7, and the value for the new window is 8.

where $h = d^{m-1} \bmod q$ is the value of the digit “1” in the high-order position of an m -digit text window.

The solution of working modulo q is not perfect, however: $t_s = p \pmod{q}$ does not automatically mean that $t_s = p$. On the other hand, if $t_s \neq p \pmod{q}$, then you definitely know that $t_s \neq p$, so that shift s is invalid. Thus you can use the test $t_s = p \pmod{q}$ as a fast heuristic test to rule out invalid shifts. If $t_s = p \pmod{q}$ —a *hit*—then you need to test further to see whether s is really valid or you just have a *spurious hit*. This additional test explicitly checks the condition $P[1:m] = T[s+1:s+m]$. If q is large enough, then you would hope that spurious hits occur infrequently enough that the cost of the extra checking is low.

The procedure RABIN-KARP-MATCHER on the next page makes these ideas precise. The inputs to the procedure are the text T , the pattern P , their lengths n and m , the radix d to use (which is typically taken to be $|\Sigma|$), and the prime q to use. The procedure works as follows. All characters are interpreted as radix- d digits. The subscripts on t are provided only for clarity: the procedure works correctly if all the subscripts are dropped. Line 1 initializes h to the value of the high-order digit position of an m -digit window. Lines 2–6 compute p as the value of $P[1:m] \bmod q$ and t_0 as the value of $T[1:m] \bmod q$. The **for** loop of lines 7–12 iterates through all possible shifts s , maintaining the following invariant:

Whenever line 8 is executed, $t_s = T[s+1:s+m] \bmod q$.

If a hit occurs because $p = t_s$ in line 8, then line 9 determines whether s is a valid shift or the hit was spurious via the test $P[1:m] == T[s+1:s+m]$. Line 10 prints out any valid shifts that are found. If $s < n - m$ (checked in line 11), then the **for** loop will iterate at least one more time, and so line 12 first executes to ensure that the loop invariant holds upon the next iteration. Line 12 computes the value of $t_{s+1} \bmod q$ from the value of $t_s \bmod q$ in constant time using equation (32.2) directly.

RABIN-KARP-MATCHER takes $\Theta(m)$ preprocessing time, and its matching time is $\Theta((n - m + 1)m)$ in the worst case, since (like the naive string-matching algorithm) the Rabin-Karp algorithm explicitly verifies every valid shift. If $P = \mathbf{a}^m$ and $T = \mathbf{a}^n$, then verifying takes $\Theta((n - m + 1)m)$ time, since each of the $n - m + 1$ possible shifts is valid.

In many applications, you expect few valid shifts—perhaps some constant c of them. In such applications, the expected matching time of the algorithm is only $O((n - m + 1) + cm) = O(n + m)$, plus the time required to process spurious hits. We can base a heuristic analysis on the assumption that reducing values modulo q acts like a random mapping from Σ^* to $\mathbb{Z}_q$. The expected number of spurious hits is then $O(n/q)$, because we can estimate the chance that an arbitrary t_s will be equivalent to p , modulo q , as $1/q$. Since there are $O(n)$ positions at which the

```

RABIN-KARP-MATCHER( $T, P, n, m, d, q$ )
1   $h = d^{m-1} \bmod q$ 
2   $p = 0$ 
3   $t_0 = 0$ 
4  for  $i = 1$  to  $m$                                 // preprocessing
5       $p = (dp + P[i]) \bmod q$ 
6       $t_0 = (dt_0 + T[i]) \bmod q$ 
7  for  $s = 0$  to  $n - m$                                 // matching—try all possible shifts
8      if  $p == t_s$                                     // a hit?
9          if  $P[1:m] == T[s+1:s+m]$  // valid shift?
10             print “Pattern occurs with shift”  $s$ 
11      if  $s < n - m$ 
12           $t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$ 

```

test of line 8 fails (actually, at most $n - m + 1$ positions) and checking each hit takes $O(m)$ time in line 9, the expected matching time taken by the Rabin-Karp algorithm is

$$O(n) + O(m(v + n/q)),$$

where v is the number of valid shifts. This running time is $O(n)$ if $v = O(1)$ and you choose $q \geq m$. That is, if the expected number of valid shifts is small ($O(1)$) and you choose the prime q to be larger than the length of the pattern, then you can expect the Rabin-Karp procedure to use only $O(n + m)$ matching time. Since $m \leq n$, this expected matching time is $O(n)$.

Exercises

32.2-1

Working modulo $q = 11$, how many spurious hits does the Rabin-Karp matcher encounter in the text $T = 3141592653589793$ when looking for the pattern $P = 26$?

32.2-2

Describe how to extend the Rabin-Karp method to the problem of searching a text string for an occurrence of any one of a given set of k patterns. Start by assuming that all k patterns have the same length. Then generalize your solution to allow the patterns to have different lengths.

32.2-3

Show how to extend the Rabin-Karp method to handle the problem of looking for a given $m \times m$ pattern in an $n \times n$ array of characters. (The pattern may be shifted vertically and horizontally, but it may not be rotated.)

32.2-4

Alice has a copy of a long n -bit file $A = \langle a_{n-1}, a_{n-2}, \dots, a_0 \rangle$, and Bob similarly has an n -bit file $B = \langle b_{n-1}, b_{n-2}, \dots, b_0 \rangle$. Alice and Bob wish to know if their files are identical. To avoid transmitting all of A or B , they use the following fast probabilistic check. Together, they select a prime $q > 1000n$ and randomly select an integer x from $\{0, 1, \dots, q-1\}$. Letting

$$A(x) = \left(\sum_{i=0}^{n-1} a_i x^i \right) \bmod q \quad \text{and} \quad B(x) = \left(\sum_{i=0}^{n-1} b_i x^i \right) \bmod q,$$

Alice evaluates $A(x)$ and Bob evaluates $B(x)$. Prove that if $A \neq B$, there is at most one chance in 1000 that $A(x) = B(x)$, whereas if the two files are the same, $A(x)$ is necessarily the same as $B(x)$. (*Hint*: See Exercise 31.4.4.)

32.3 String matching with finite automata

Many string-matching algorithms build a finite automaton—a simple machine for processing information—that scans the text string T for all occurrences of the pattern P . This section presents a method for building such an automaton. These string-matching automata are efficient: they examine each text character *exactly once*, taking constant time per text character. The matching time used—after pre-processing the pattern to build the automaton—is therefore $\Theta(n)$. The time to build the automaton, however, can be large if Σ is large. Section 32.4 describes a clever way around this problem.

We begin this section with the definition of a finite automaton. We then examine a special string-matching automaton and show how to use it to find occurrences of a pattern in a text. Finally, we'll see how to construct the string-matching automaton for a given input pattern.

Finite automata

A *finite automaton* M , illustrated in Figure 32.5, is a 5-tuple $(Q, q_0, A, \Sigma, \delta)$, where

- Q is a finite set of *states*,
- $q_0 \in Q$ is the *start state*,
- $A \subseteq Q$ is a distinguished set of *accepting states*,
- Σ is a finite *input alphabet*,
- δ is a function from $Q \times \Sigma$ into Q , called the *transition function* of M .

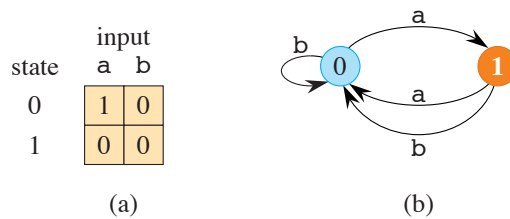


Figure 32.5 A simple two-state finite automaton with state set $Q = \{0, 1\}$, start state $q_0 = 0$, and input alphabet $\Sigma = \{a, b\}$. **(a)** A tabular representation of the transition function δ . **(b)** An equivalent state-transition diagram. State 1, in orange, is the only accepting state. Directed edges represent transitions. For example, the edge from state 1 to state 0 labeled b indicates that $\delta(1, b) = 0$. This automaton accepts those strings that end in an odd number of a 's. More precisely, it accepts a string x if and only if $x = yz$, where $y = \varepsilon$ or y ends with a b , and $z = a^k$, where k is odd. For example, on input $abaaaa$, including the start state, this automaton enters the sequence of states $\langle 0, 1, 0, 1, 0, 1 \rangle$, and so it accepts this input. For input $abbbaa$, it enters the sequence of states $\langle 0, 1, 0, 0, 1, 0 \rangle$, and so it rejects this input.

The finite automaton begins in state q_0 and reads the characters of its input string one at a time. If the automaton is in state q and reads input character a , it moves (“makes a transition”) from state q to state $\delta(q, a)$. Whenever its current state q is a member of A , the machine M has *accepted* the string read so far. An input that is not accepted is *rejected*.

A finite automaton M induces a function ϕ , called the *final-state function*, from Σ^* to Q such that $\phi(w)$ is the state M ends up in after reading the string w . Thus, M accepts a string w if and only if $\phi(w) \in A$. We define the function ϕ recursively, using the transition function:

$$\begin{aligned}\phi(\varepsilon) &= q_0, \\ \phi(wa) &= \delta(\phi(w), a) \quad \text{for } w \in \Sigma^*, a \in \Sigma.\end{aligned}$$

String-matching automata

For a given pattern P , a preprocessing step constructs a string-matching automaton specific to P . The automaton then searches the text string for occurrences of P . Figure 32.6 illustrates the automaton for the pattern $P = ababaca$. From now on, let's assume that P is fixed, and for brevity, we won't bother to indicate the dependence upon P in our notation.

In order to specify the string-matching automaton corresponding to a given pattern $P[1:m]$, we first define an auxiliary function σ , called the *suffix function* corresponding to the pattern P . The function σ maps Σ^* to $\{0, 1, \dots, m\}$ such that $\sigma(x)$ is the length of the longest prefix of P that is also a suffix of x :

$$\sigma(x) = \max \{k : P[1:k] \sqsubset x\} . \quad (32.3)$$